

SkyCast: Complete File-by-File Architecture Guide

An Exhaustive Technical Dictionary of Every File: What, How, Why, and When

Document Metadata

- **Project Name:** SkyCast (Hyperlocal Weather & Algorithmic Outdoor Activity Planner)
- **Author / Lead Developer:** Laxman Patel
- **Repository:** <https://github.com/laxman-patel-007/SkyCast-Weather-Planner>
- **Framework & Toolchain:** Flutter 3.44.8 • Dart 3.12.2 (Sound Null-Safety)
- **Architecture Pattern:** Clean Layered Architecture (Presentation • State • Service • Data)
- **Target Platforms:** Responsive Web (Chrome/Edge/Safari), iOS, Android, macOS

1. Directory Structure Map

lib/	
├─ main.dart	# Application Bootstrap & Provider Injection
├─ core/	
│ └─ theme/	
│ └─ app_theme.dart	# Material 3 Light/Dark Themes & Color Tokens
│ └─ weather_gradients.dart	# Dynamic Atmospheric Gradient Generator
├─ models/	
│ └─ weather_condition.dart	# Weather Enum with Icons & Gradient Getters
│ └─ hourly_forecast.dart	# 1-Hour Telemetry Entity & JSON Serialization
│ └─ weather_forecast.dart	# 24-Hour Aggregate Entity & CopyWith Methods
│ └─ activity_suggestion.dart	# Activity Scoring Entity & Suitability Badges
├─ providers/	
│ └─ theme_provider.dart	# Light/Dark State Management & Disk Persistence
│ └─ weather_provider.dart	# Weather State, Selected Hour & Rule Execution
├─ screens/	
│ └─ main_navigation_screen.dart	# Responsive Root Shell (Web SaaS Header / Mobile
│ └─ home_forecast_screen.dart	# Hero Card, 24h Carousel, Top Activities, Sidebar
│ └─ hourly_detail_screen.dart	# 24h Timeline Inspector & Drill-down Modal
│ └─ activity_suggestions_screen.dart	# 10 Activity Cards, 8 Category Chips, Context Bar
├─ services/	
│ └─ weather_service.dart	# 8 Microclimate Stations & Diurnal Math Simulation
│ └─ weather_cache_service.dart	# SharedPreferences Offline Storage Strategy
│ └─ activity_planner_service.dart	# 10-Activity Physics & Physiology Rule Engine
├─ widgets/	
│ └─ activity_card_widget.dart	# Expandable Activity Tile with Suitability Bars
│ └─ cache_indicator_badge.dart	# Online/Cache Status Chip & Test Trigger
│ └─ desktop_web_header.dart	# Sleek SaaS Top Navigation Bar for Web Viewports
│ └─ hourly_forecast_card.dart	# Horizontal Carousel & Vertical List Hourly Cards
│ └─ location_selector_sheet.dart	# Hyperlocal Station Picker Modal Sheet
│ └─ metric_chip.dart	# Modular Atmospheric Telemetry Chip
│ └─ weather_gradient_card.dart	# Dynamic Hero Weather Card with 8 Telemetry Chips
test/	
├─ skycast_test.dart	# Unit Tests: Models, JSON Serialization, Algorithmic
├─ widget_test.dart	# Component Tests: App Composition & Navigation
└─ mobile_device_test.dart	# Responsive Tests: 5 Form Factors (SE to Fold)

2. Core Entrypoint & Bootstrap

`lib/main.dart`

- **What it does:** The application entrypoint. Initializes Flutter platform bindings, injects top-level providers, and mounts `SkyCastApp`.
 - **How it works:**
 - Calls `WidgetsFlutterBinding.ensureInitialized()` to ensure native communication channels are ready.
 - Sets up `MultiProvider` hosting `ThemeProvider` and `WeatherProvider`.
 - In `WeatherProvider.initialize()`, immediately initiates the offline cache-first hydration pipeline.
 - Binds `MaterialApp` theme to `themeProvider.themeMode`, alternating between `AppTheme.lightTheme()` and `AppTheme.darkTheme()`.
 - **Why it exists:** Provides a single, clean root configuration where dependency injection and global lifecycle initialization occur before any UI widget renders.
 - **When it is executed:** Executed exactly once when the operating system launches the application binary or when the browser loads `index.html`.
-

3. Core Theming System (`lib/core/theme/`)

`lib/core/theme/app_theme.dart`

- **What it does:** Defines the global Material 3 theme configurations for both Light and Dark modes, providing semantic color accessors.
 - **How it works:**
 - Configures `ThemeData(useMaterial3: true)` with seed colors (`0xFF0284C7` Sky Blue).
 - Exposes static helper functions: `AppTheme.cardColor(context)`, `AppTheme.textPrimary(context)`, `AppTheme.textSecondary(context)`, and `AppTheme.borderColor(context)`.
 - Dynamically evaluates `MediaQuery.of(context).platformBrightness` or active theme settings via `AppTheme.isDark(context)`.
 - **Why it exists:** Centralizes visual styling into a single design system. Prevents hardcoding random color hexes in widgets, ensuring WCAG-compliant contrast ratios across the app.
 - **When it is executed:** Evaluated during the build phase of every widget that queries theme properties.
-

`lib/core/theme/weather_gradients.dart`

- **What it does:** Calculates dynamic multi-stop linear background gradients corresponding to real-time meteorological conditions.
- **How it works:**
 - Implements `WeatherGradients.getGradientForCondition(WeatherCondition condition, {bool isDark = false})`.
 - Maps conditions to harmonious palettes:

- `sunny` : Sunrise Orange (`#FF7E40`) $\rightarrow$ Golden Amber (`#FFB347`) $\rightarrow$ Sky Blue (`#4A90E2`).
 - `snowy` : Alpine Frost (`#4B6CB7`) $\rightarrow$ Deep Polar Navy (`#182848`) $\rightarrow$ Obsidian (`#000428`).
 - `heatWave` : Crimson (`#DC2626`) $\rightarrow$ Solar Orange (`#EA580C`) $\rightarrow$ Thermal Amber (`#991B1B`).
 - `foggy` : Cool Slate (`#4A5568`) $\rightarrow$ Shadow Steel (`#2D3748`) $\rightarrow$ Heavy Mist (`#1A202C`).
 - **Why it exists:** Provides contextual immersion. Users immediately understand the atmospheric condition outdoors via subconscious color language before reading numerical metrics.
 - **When it is executed:** Called whenever `WeatherGradientCard` builds or re-renders upon weather state changes.
-

4. Domain & Data Models (`lib/models/`)

`lib/models/weather_condition.dart`

- **What it does:** Enumerates all 12 supported meteorological conditions and attaches display metadata.
 - **How it works:**
 - Dart 3 enhanced enum defining: `sunny`, `partlyCloudy`, `cloudy`, `rainy`, `heavyRain`, `thunderstorm`, `snowy`, `foggy`, `hail`, `heatWave`, `windy`, and `clearNight`.
 - Exposes properties: `displayName` (formatted title), `icon` (Material icon), and `gradientColors` (color stops).
 - **Why it exists:** Replaces error-prone raw string literals (`"rain"`, `"rainy"`) with compile-time type safety.
 - **When it is executed:** Referenced throughout the domain, service, and UI layers whenever weather conditions are parsed, compared, or rendered.
-

`lib/models/hourly_forecast.dart`

- **What it does:** Represents a discrete 1-hour meteorological telemetry snapshot (temperature, rain chance, wind speed, pressure, visibility).
 - **How it works:**
 - Immutable class with `final` fields and a `const` constructor.
 - Implements `toJson()` returning `Map<String, dynamic>` and `HourlyForecast.fromJson()` factory constructor with robust fallback defaults.
 - Provides getters: `formattedTime`, `formattedTemp`, `formattedPressure`, `formattedVisibility`.
 - **Why it exists:** Serves as the atomic building block for the 24-hour forecast timeline and hourly activity evaluations.
 - **When it is executed:** Instantiated by `WeatherService` when simulating diurnal timelines, and serialized/deserialized by `WeatherCacheService`.
-

`lib/models/weather_forecast.dart`

- **What it does:** Aggregate root model representing the complete 24-hour hyperlocal forecast for a specific location.
 - **How it works:**
 - Encapsulates `locationName`, `latitude`, `longitude`, `elevationMeters`, `fetchedException`, and a `List<HourlyForecast> hourlyForecasts`.
 - Implements JSON serialization (`toJson`, `fromJson`) and an immutable `copyWith(...)` method.
 - Exposes convenience getters for current temperature, daily high/low, current humidity, wind speed, and air quality index (AQI).
 - **Why it exists:** Consolidates all weather parameters into a single immutable data structure passed across providers and UI screens.
 - **When it is executed:** Created by `WeatherService`, held in memory by `WeatherProvider`, and written to `SharedPreferences` by `WeatherCacheService`.
-

`lib/models/activity_suggestion.dart`

- **What it does:** Models an outdoor activity's suitability evaluation, scoring match, and scientific justifications.
 - **How it works:**
 - Holds `activityId`, `name`, `category`, `score` (0.0 to 100.0), `suitability` enum, `justification`, `bestTimeWindow`, `thresholds`, and `gearTips`.
 - `SuitabilityLevel` enum defines: `ideal` (80–100%), `good` (60–79%), `moderate` (40–59%), and `caution` (0–39%).
 - Provides color and badge helpers: `badgeColor`, `badgeTextColor`, `icon`.
 - **Why it exists:** Connects raw meteorological measurements to actionable human recommendations.
 - **When it is executed:** Generated dynamically by `ActivityPlannerService` whenever weather conditions update or when a user selects a specific hour.
-

5. State Management Providers (`lib/providers/`)

`lib/providers/theme_provider.dart`

- **What it does:** Manages global Light/Dark theme mode state and persists the user's preference to local storage.
 - **How it works:**
 - Extends `ChangeNotifier`.
 - On initialization, reads `skycast_theme_mode` (`'light'` or `'dark'`) from `SharedPreferences`.
 - `toggleTheme()` toggles between `ThemeMode.light` and `ThemeMode.dark`, persists the change, and calls `notifyListeners()`.
 - **Why it exists:** Decouples theme state from individual widgets, allowing the entire application tree to rebuild reactively upon a toggle.
 - **When it is executed:** Initialized on app boot; invoked whenever the user taps the Sun/Moon toggle button in the desktop header or mobile AppBar.
-

`lib/providers/weather_provider.dart`

- **What it does:** The central state container for the application. Manages weather fetching, selected microclimates, active hour inspector, offline cache state, and activity recalculation.
 - **How it works:**
 - Extends `ChangeNotifier`.
 - `initialize()` : Executes the cache-first hydration strategy (reads disk cache immediately, renders UI, then fetches fresh data).
 - `setLocation(presetId)` : Updates active microclimate station and triggers an asynchronous fetch.
 - `selectHour(hourly)` : Highlights an hour on the timeline and triggers `_recalculateActivities()` for that hour's conditions.
 - `refresh()` : Forces a fresh fetch from `WeatherService` and writes the result to `WeatherCacheService`.
 - **Why it exists:** Serves as the Single Source of Truth for the application, ensuring all screens stay synchronized without tight coupling.
 - **When it is executed:** Active throughout the app's lifecycle, reacting to user interactions, network responses, and timer events.
-

6. Service & Business Logic Layer (`lib/services/`)

`lib/services/weather_service.dart`

- **What it does:** Simulates meteorological weather stations and generates realistic 24-hour diurnal timelines across 8 diverse microclimates.
 - **How it works:**
 - Defines 8 presets: *Central Green Valley*, *Coastal Bay Marina*, *Highland Ridge*, *Sunny Vista*, *Alpine Pine Summit*, *Lakeside Wetland*, *Desert Oasis*, and *Stargazer Hill*.
 - Uses diurnal sine wave mathematics to calculate realistic hourly temperatures, barometric pressures, humidity curves, and solar UV indices.
 - Simulates non-blocking asynchronous network latency (`Future.delayed(650ms)`).
 - **Why it exists:** Provides consistent, high-fidelity meteorological data for offline demonstration and testing without requiring third-party API keys or rate-limiting risks.
 - **When it is executed:** Called by `WeatherProvider` when booting, refreshing, or switching microclimate stations.
-

`lib/services/weather_cache_service.dart`

- **What it does:** Handles offline local persistence using `SharedPreferences`.
- **How it works:**
 - `saveForecast(forecast)` : Serializes `WeatherForecast` to JSON string and writes to disk with timestamp.
 - `getCachedForecast()` : Reads the stored JSON string, decodes it, and reconstructs the `WeatherForecast` entity.
 - `getLastSyncTime()` : Returns the formatted human-readable time of the last successful synchronization.

- **Why it exists:** Implements the Cache-Aside pattern, satisfying the zero-blank-screen requirement during cold boots or offline network scenarios.
 - **When it is executed:** Called during app startup (`initialize()`) and immediately after every successful fresh weather fetch.
-

`lib/services/activity_planner_service.dart`

- **What it does:** Algorithmic evaluation engine calculating suitability scores (0–100%) and plain-language scientific justifications for 10 outdoor activities.
 - **How it works:**
 - Evaluates: *Jogging, Picnic, Cycling, Hiking, Kayaking, Tennis, Stargazing, Yoga, Bouldering, and Photography.*
 - Evaluates temperature, wind resistance, precipitation, humidity, UV index, and visibility against physiological and physical limits.
 - Produces structured `ActivitySuggestion` instances with optimal time windows and gear preparation tips.
 - **Why it exists:** Delivers the core value proposition of the capstone project: translating raw atmospheric telemetry into actionable human planning decisions.
 - **When it is executed:** Invoked automatically by `WeatherProvider` whenever fresh weather data arrives or when a specific forecast hour is selected.
-

7. UI Presentation Screens (`lib/screens/`)

`lib/screens/main_navigation_screen.dart`

- **What it does:** The root presentation shell hosting navigation controls and managing screen switching.
 - **How it works:**
 - Uses an `IndexedStack` to preserve state across tabs (Home, Hourly, Activities).
 - Inspects screen width via `MediaQuery` :
 - **Desktop (≥960px):** Renders the custom `DesktopWebHeader` at the top and hides the bottom navigation bar.
 - **Mobile (<960px):** Renders the Material 3 `NavigationBar` at the bottom.
 - **Why it exists:** Provides responsive platform adaptation from a single codebase while preventing tab re-render loss during navigation.
 - **When it is executed:** Built immediately after `main.dart` finishes bootstrap, remaining active throughout the session.
-

`lib/screens/home_forecast_screen.dart`

- **What it does:** The primary dashboard screen displaying current weather, 24-hour glance carousel, top activity highlights, and microclimate stations.
- **How it works:**
 - Uses `LayoutBuilder` to adapt between a 2-column SaaS web dashboard (wide viewports) and a single-column mobile stream.

- Integrates `CacheIndicatorBadge` , `WeatherGradientCard` , `HourlyForecastCard` , and preview activity cards.
 - Supports pull-to-refresh gestures.
 - **Why it exists:** Serves as the landing hub of the application, delivering an immediate overview of current conditions and top recommendations.
 - **When it is executed:** Displayed whenever Tab 0 (`Home`) is active.
-

`lib/screens/hourly_detail_screen.dart`

- **What it does:** Detailed 24-hour chronological timeline screen with atmospheric parameter inspector.
 - **How it works:**
 - Features a sticky top inspector card showing detailed metrics for the currently selected hour.
 - Renders a vertical `ListView.builder` displaying all 24 hours with temperature bars, wind speeds, rain risks, pressure, and visibility.
 - Tapping any hour selects it, re-evaluating activities across the app.
 - **Why it exists:** Allows users to investigate temporal changes throughout the day to plan activities around specific weather windows.
 - **When it is executed:** Displayed whenever Tab 1 (`Hourly (24h)`) is active or when tapped from the home carousel.
-

`lib/screens/activity_suggestions_screen.dart`

- **What it does:** Comprehensive outdoor activity planner screen with category filtering and detailed recommendation cards.
 - **How it works:**
 - Displays an active context banner indicating whether ratings reflect current weather or a selected future hour (with a reset button).
 - Provides 8 horizontal category filter chips (*All, Cardio & Fitness, Leisure & Family, Active Transit & Sport, Outdoor Adventure, Water & Aquatics, Night & Astronomy, Wellness & Mindfulness, Creative Arts*).
 - Renders all matching activity cards in a responsive grid/list.
 - **Why it exists:** Dedicated interface for browsing, filtering, and studying activity suitability and gear tips.
 - **When it is executed:** Displayed whenever Tab 2 (`Activities`) is active.
-

8. Modular UI Widgets (`lib/widgets/`)

`lib/widgets/desktop_web_header.dart`

- **What it does:** SaaS-style top navigation header displayed on wide desktop browser viewports.
- **How it works:**
 - Renders branding logo, application title, navigation pills (`Home` , `Hourly` , `Activities`), Theme Toggle button, and Refresh button.
 - Highlights active tab using subtle animated background containers.

- **Why it exists:** Replaces awkward mobile bottom bars on desktop viewports, giving the web app a modern, desktop-first aesthetic.
 - **When it is executed:** Built at the top of `MainNavigationScreen` whenever screen width $\geq 960\text{px}$.
-

`lib/widgets/weather_gradient_card.dart`

- **What it does:** The hero weather card displaying temperature, location, condition, high/low, and 8 atmospheric telemetry metrics.
 - **How it works:**
 - Background dynamically generated via `WeatherGradients`.
 - Displays location badge, 68pt temperature typography, high/low bar, and 8 `MetricChip` widgets: Wind, Rain Risk, Humidity, Air Quality (AQI), Barometer, Visibility, UV Index, and Dew Point.
 - **Why it exists:** Serves as the primary visual anchor of the home screen, providing rich atmospheric telemetry at a glance.
 - **When it is executed:** Embedded inside `HomeForecastScreen`.
-

`lib/widgets/metric_chip.dart`

- **What it does:** Reusable glassmorphic chip displaying an individual atmospheric metric (icon, title, value, subtitle).
 - **How it works:**
 - Encapsulates icon, label, primary value, and optional qualitative subtitle in a rounded container with subtle border strokes.
 - **Why it exists:** Eliminates code duplication across the 8 telemetry metrics on the hero card and hourly inspector.
 - **When it is executed:** Rendered within `WeatherGradientCard` and `HourlyDetailScreen`.
-

`lib/widgets/hourly_forecast_card.dart`

- **What it does:** Renders a 1-hour forecast tile in either a compact horizontal carousel variant or an expanded vertical list variant.
 - **How it works:**
 - Displays hour label, condition icon, bold temperature, and precipitation chance badge.
 - Highlights active state with animated border and background glow when selected.
 - **Why it exists:** Reusable component serving both the quick-glance carousel on the home screen and the 24-hour detail screen.
 - **When it is executed:** Built inside the horizontal scroll view on Home and the vertical list on Hourly Detail.
-

`lib/widgets/activity_card_widget.dart`

- **What it does:** Detailed card displaying an outdoor activity's suitability score, progress bar, justification, time window, and gear tips.
- **How it works:**

- Visualizes score (0–100%) via a color-coded `LinearProgressIndicator` .
 - Displays suitability badge (`Ideal` , `Good` , `Moderate` , `Caution`), meteorological justification, threshold chips, and prep notes.
 - **Why it exists:** Standardizes activity presentation across the home screen previews and the activities screen.
 - **When it is executed:** Built inside `ActivitySuggestionsScreen` and the home screen recommendation section.
-

`lib/widgets/cache_indicator_badge.dart`

- **What it does:** Displays current data freshness status (Live Hyperlocal vs. Local Storage Cache) and provides an offline test button.
 - **How it works:**
 - Reads `provider.isFromCache` and `provider.lastSyncTime` .
 - Renders a pulsing green/amber status indicator dot and a "Test Cache" button that triggers simulated offline re-hydration.
 - **Why it exists:** Fulfills case study transparency requirements by proving offline caching functionality to evaluators.
 - **When it is executed:** Built at the top of `HomeForecastScreen` .
-

`lib/widgets/location_selector_sheet.dart`

- **What it does:** Bottom modal sheet allowing users to switch between the 8 hyperlocal microclimate stations.
 - **How it works:**
 - Renders a scrollable list of all 8 preset stations with descriptions, icons, elevations, and temperatures.
 - Highlights current station; tapping a station invokes `onSelected(id)` and closes the modal.
 - **Why it exists:** Allows users to explore different microclimate topographies without needing GPS permissions.
 - **When it is executed:** Opened when tapping location chips on the hero weather card.
-

9. Automated Testing Suite (`test/`)

`test/skycast_test.dart`

- **What it does:** Comprehensive unit test suite validating domain entities, JSON serialization, cache logic, and activity scoring algorithms.
 - **How it works:**
 - Tests `HourlyForecast` JSON roundtrip fidelity.
 - Tests `WeatherCondition` parsing and safe fallbacks.
 - Asserts that all 10 outdoor activities score appropriately under cold, rainy, windy, and ideal conditions.
 - **Why it exists:** Guarantees core business logic remains correct and free of regression bugs during refactoring.
 - **When it is executed:** Run during CI/CD or via terminal: `flutter test test/skycast_test.dart` .
-

test/widget_test.dart

- **What it does:** Widget and component test suite verifying root widget composition, navigation destinations, and theme switching.
 - **How it works:**
 - Uses `WidgetTester` to pump `SkyCastApp`.
 - Verifies presence of navigation destinations (`Home` , `Hourly` , `Activities`).
 - Asserts theme toggling updates the widget tree.
 - **Why it exists:** Validates that UI components mount and respond to state changes without crashing.
 - **When it is executed:** Run via `flutter test test/widget_test.dart`.
-

test/mobile_device_test.dart

- **What it does:** Responsive multi-device layout test matrix validating rendering across 5 physical phone sizes.
 - **How it works:**
 - Iterates through: iPhone SE (`320x568`), Standard Android (`360x640`), iPhone X (`375x812`), iPhone 15 Pro (`390x844`), and Pixel 7 (`412x915`).
 - Pumps the app at each resolution and asserts `tester.takeException() == null`.
 - **Why it exists:** Prevents `RenderFlex overflowed` errors across small, narrow, or tall phone form factors.
 - **When it is executed:** Run via `flutter test test/mobile_device_test.dart`.
-

10. Project Configuration Files

pubspec.yaml

- **What it does:** Project manifest declaring dependencies, SDK constraints, asset configurations, and fonts.
 - **How it works:**
 - Dependencies: `flutter` , `provider: ^6.1.5+1` , `shared_preferences: ^2.5.5` , `intl: ^0.20.3` .
 - Dev dependencies: `flutter_test` , `flutter_lints: ^5.0.0` .
 - Configuration: `uses-material-design: true` .
 - **Why it exists:** Required by Flutter and Dart to resolve packages, enforce SDK compatibility, and manage builds.
 - **When it is executed:** Evaluated during `flutter pub get` , `flutter build` , and `flutter run` .
-

analysis_options.yaml

- **What it does:** Static analysis configuration defining compiler linting rules and code style guidelines.
- **How it works:**
 - Extends `package:flutter_lints/flutter.yaml` .
 - Enforces sound null safety, const constructors, and clean coding standards.

- **Why it exists:** Guarantees high code quality across the team, validated by `dart analyze` reporting 0 issues.
 - **When it is executed:** Run continuously inside the IDE and during `dart analyze` .
-

Document compiled and verified for the SkyCast Capstone Project.